EviRecall

Evidence-Aware Memory for Long-Horizon Multi-Agent Tasks

Research proposal v0.3 | 28 October 2026 | CSC5051 / CSC6052 / MDS5110

Project repository: github.com/SatouJotaro/evirecall

Member	Full name	Student ID	University email
A	[Member A name]	[Member A ID]	[Member A email]
B	[Member B name]	[Member B ID]	[Member B email]
C	[Member C name]	[Member C ID]	[Member C email]

Personal-information placeholders are blue in the PDF and editable LaTeX. Replace them before submission.

Project summary. We propose to extend Smurfs with evidence-aware shared memory and investigate whether checking recalled evidence prevents stale information from affecting later decisions. Our contribution will combine one memory intervention, controlled evidence/error experiments, and a reproducible benchmark harness. This is a research plan; no performance improvements are claimed.

1 Motivation and research questions

Multi-agent tasks depend on information passed between planning, execution, answering, and verification roles. Compressed memory can omit a decisive constraint. Even a correctly cited summary can become outdated after the user or environment supplies new evidence. Reusing that summary may propagate an error across roles and subsequent tasks.

For example, a travel-planning agent may retain a plan after a user reduces the budget. Later roles might reuse that plan and verify only its final wording, without revisiting the changed constraint. This is an illustrative scenario, not an experimental observation.

- **RQ1 - Memory utility:** Does tiered memory improve recall and subsequent task completion over recent history and ordinary retrieval?
- **RQ2 - Evidence and recovery:** Does checking evidence validity reduce stale-information reuse across roles and improve recovery after an update?
- **RQ3 - Resource trade-offs:** How do evidence budgets and task concurrency affect quality, cost, and completion time?

Current choice. Use Smurfs as the fixed multi-agent backbone; pilot Jev TierMem as the first memory backend; evaluate general memory with LongMemEval and stale-memory behavior with STALE. Add a small MemoryArena action study if its environment passes the feasibility pilot. MAST informs failure annotation. No parameter training is required in the first version.

Expected outputs. A reusable memory adapter and intervention; source-linked task traces and error analysis; reproducible comparisons and resource measurements. The following sections document the alternatives considered and how this scope was chosen.

2 Multi-agent and memory selection

We reviewed the downloaded papers, official repositories, and relevant interfaces. Selection favors task fit, visible integration points, reproducible evidence, and a feasible three-person workload. This review is not a claim that every candidate has been reproduced.

2.1 Multi-agent systems

Candidate and code	Relevant capability and trade-off	Decision
Smurfs [1]	Explicit planning, execution, answering, and verification; inspectable tool loop. Persistent shared memory must be added.	**Selected backbone.** Fits the topic and the instructor's research.
Deep Agents / LangGraph [8]	Subagents, context/file tools, and persistence primitives reduce engineering work. They introduce a different orchestration stack.	Engineering alternative; no second backbone in version 1.
Magentic-One [11]	General web/file/code roles support broad workflows, adding environment setup beyond our first study.	Architecture reference; current AutoGen source is archived.
TwinMarket [12]	Multi-agent financial and social simulation introduces domain-specific tasks and evaluation.	Reviewed; outside the selected task scope.
EasyMED [13]	Multi-agent patient simulation and educational evaluation require medical task definitions.	Reviewed; outside the selected task scope.

Why Smurfs: its role boundaries let us observe where evidence enters, is summarized, is reused, and is verified. We will keep this backbone fixed while changing memory behavior.

2.2 Memory mechanisms

Mechanism and source	Relevant capability and trade-off	Decision
Recent history / BM25	Simple controls: limited context versus retrieval over raw chunks. Neither explicitly tracks superseded conclusions.	Required low-complexity baselines.
Jev TierMem [3]	Lightweight summary/raw-evidence storage with a judgment service; exposes Python integration points.	**First backend:** small interface, no router training. Pilot service access.
TierMem paper implementation [2]	Provenance-linked tiers and trained routing; reproducing its training/service stack adds work.	Method reference; distinct from Jev, not a second mandatory build.
CUP-Mem [10]	State revision and propagation-aware updates directly overlap our proposed intervention.	Closest method comparator on a matched STALE subset.
A-MEM [14] / Mem0 [15]	Dynamic memory organization, links, and updates offer additional designs and dependencies.	Related methods; not required in the initial comparison.
Governed Shared Memory / Caura [7]	Temporal supersession and provenance across agents; current service code differs from the paper artifact.	Systems reference; no full service deployment planned.

3 Dataset and benchmark selection

We separate memory-question answering, action execution, and retrospective failure analysis. A dataset with long histories does not automatically evaluate multi-agent collaboration or successful actions.

Resource and code	What it evaluates; relevant limitation	Use in EviRecall
LongMemEval [4]	500 questions covering extraction, cross-session reasoning, updates, time, and abstention. Primarily memory QA.	**Primary:** cleaned S release for general memory utility.
STALE [10]	400 scenarios, three queries each: state resolution, premise resistance, and policy adaptation. Tests implicit conflicts.	**Primary:** stale-memory behavior and CUP-Mem comparison.
MemoryArena [5]	Interdependent multi-session actions. Travel tools require an additional database; code is marked preview.	Small action pilot, then a limited travel study if feasible.
MAST / MAST-Data [6]	1,600+ annotated traces and 14 failure modes. Traces are not replayable Smurfs tasks.	Annotation reference, not a memory-training dataset.
MemoryAgentBench [9]	Incremental memory evaluation, including conflict resolution; some sources overlap LongMemEval.	Fallback for memory testing, not a substitute for action evidence.
LoCoMo [17]	Long conversational memory; useful for comparison with TierMem. Less direct coverage of our failure question.	Optional cross-check; excluded from the minimum experiment set.

Resulting plan: two primary memory evaluations plus one action pilot, using existing releases. These are not a training/test pair. We will reserve disjoint development data for prompts and thresholds; the initial method does not train model parameters. STALE uses LongMemEval-derived distractors, so overlap must be audited and cross-dataset dependence disclosed.

4 What we will build

A. Memory integration and one intervention. Connect Smurfs to a replaceable adapter with immutable source IDs and typed observations. Before reusing a summary affected by an explicit, authoritative update, check its validity, mark dependent conclusions for review, and recover raw evidence within budget. Preserve old records; ambiguous changes remain uncertain. Provenance and invalidation are existing ideas; our contribution is their controlled study at Smurfs role handoffs. STALE tests generalization to implicit conflicts without gold update labels.

B. Evidence and error experiments. Paired update/no-update variants form a separate stress suite, not official MemoryArena scores. Track stale-evidence reads, reuse across roles, and recovery. Use STALE and the travel pilot, with evidence-support labels and a MAST-informed error rubric.

C. A shared benchmark harness. Run matched conditions, isolate stores, collect traces and costs, and recompute metrics from saved outputs. Add a limited budget/concurrency sweep.

The proposed adapter uses `observe()`, `add_summary()`, and `retrieve()`; write-back via `promote()` is explicit. Smurfs generates answers and verifies them against the returned evidence.

5 Experimental design

5.1 Comparisons and ablations

- **Memory QA:** recent history, BM25 over identical raw chunks, unmodified Jev, and Jev plus our intervention. Fix the answer model; disclose additional models.
- **Closest method:** CUP-Mem on a matched STALE subset, subject to pilot feasibility. If unavailable, disclose the missing comparison and narrow claims.
- **Actions:** Smurfs without persistent memory, with Jev, and with our intervention; hold tools, observations, models, and limits constant.
- **Ablations:** disable validity filtering or raw-evidence recovery separately. Claims about multi-agent benefits require a budget-matched single-agent control.

5.2 Data isolation and experimental scale

Split by complete history, scenario, or task group. Gold answers, evidence-location labels, conflict explanations, and scoring state remain evaluator-only. Retain intended multi-session history but isolate stores across groups and conditions; fix query-level write-back policy. LongMemEval S, M, and oracle are context settings, not splits.

Pilot 10 development units per primary dataset and 3 travel groups. Provisionally target 100 held-out LongMemEval questions, 100 STALE scenarios, and 20 travel groups after cost estimation. Freeze IDs, prompts, models, judge, and budgets. Report coverage and counts; aim for three repetitions where affordable, with uncertainty grouped by history/scenario/task.

5.3 Quality, evidence, and recovery metrics

Use official scorers, category-level results, and human checks. Distinguish citation existence, evidence recall, and semantic support. For update cases:

- **Stale-use rate:** stale-dependent decisions / eligible post-update decisions.
- **Cross-role propagation rate:** update episodes with stale evidence reused by another role / update episodes.
- **Recovery rate:** corrected, successful episodes / update episodes.

Report denominators, failures, reviewer disagreement, and representative successful and failed traces.

5.4 Resource benchmark

Vary evidence budget and concurrency (1, 2, 4) separately on a fixed subset, holding tools and retry policy constant.

Count all role, summary, retrieval, Jev, and retry calls, including failed runs. Separate ingestion, execution, and evaluation-judge costs. Report tokens, requests, throughput, and latency including queue time; unknown usage is not zero. Label injected delays as synthetic. API results do not establish GPU utilization or hardware-scheduling performance.

6 Work allocation and collaboration

Each member owns implementation, experiments, analysis, and writing. Proposed boundaries keep the packages independently testable while making one integrated project.

Owner	Main implementation and experiment responsibility	Report contribution
A - Memory	Memory backends, minimal Smurfs read/write and verifier hooks, one update intervention, and mechanism ablations.	Method, integration design, and ablation results.
B - Evidence and errors	STALE input/label contract, one travel-tool adapter, paired scenarios, evidence-support rubric, and error/recovery analysis.	Task design, evidence analysis, and qualitative results.
C - Benchmark	Common runner, memory-QA loading, configuration/split manifests, score wrappers, usage/latency logging, and resource experiments.	Shared experimental setup, efficiency results, and reproducibility.

Independent development. Agree on task, memory, evidence, trace, and result interfaces first. A develops against fixtures, B scores fixed traces, and C runs a dummy backend. Integrate in week 1 and cross-review packages; C does not implement all baselines or environments.

Balanced scope. A owns one intervention, B one action environment, and C one runner with two resource axes. After the pilot, rebalance a bounded adapter or scorer if needed; equal effort is a planning target.

Report assembly. Each member writes their experimental sections from one frozen protocol. Jointly edit the introduction, terminology, conclusions, limitations, and contributions. Integrate early and interpret the combined results together.

7 Milestones, risks, and deliverables

Stage	Team milestone and acceptance condition
Week 1: feasibility	Agree on interfaces; run small Smurfs/Jev, STALE/CUP-Mem, and travel pilots. Verify service access, cost, lawful inputs, and one complete trace; freeze scope.
Weeks 2-3: development	Complete baselines, one intervention, task cases, and shared logging. Refine only on development data; freeze test manifests and protocol.
Week 4: evaluation	Run paired comparisons, ablations, and the limited resource sweep. Preserve failures, raw outputs, configurations, and scoring records.
Week 5: delivery	Cross-reproduce key results; assemble the report, poster, and demonstration; verify all contribution claims against artifacts.

These stages are not official deadlines. Freeze provider access, cost limits, and action scope after the pilot. An unsuccessful action pilot limits conclusions to memory and controlled cross-role tests. Disclose any Jev replacement explicitly.

Deliver code, versioned configurations/task cases, source-linked traces, analysis, and reproduction instructions. Submit a LaTeX report with individual contributions, GitHub source, and a poster presentation [16]. Report positive or negative results; follow the confirmed course format.

References

[1] Junzhi Chen, Juhao Liang, Benyou Wang. Smurfs: Multi-Agent System using Context-Efficient DFSDT for Tool Planning. 2025. NAACL, pp. 3281-3298. [Paper] [Code]

[2] Qiming Zhu, Shunian Chen, Rui Yu et al.. From Lossy to Verified: A Provenance-Aware Tiered Memory for Agents. 2026. arXiv:2602.17913. [Paper] [Code]

[3] FreedomIntelligence. Jev TierMem: Lightweight Agent Memory. 2026. Software documentation; accessed 2026-10-27. [Code]

[4] Di Wu, Hongwei Wang, Wenhao Yu et al.. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. 2025. arXiv:2410.10813. [Paper] [Code] [Data]

[5] Zexue He, Yu Wang, Churan Zhi et al.. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. 2026. arXiv:2602.16313. [Paper] [Code] [Data]

[6] Mert Cemri, Melissa Z. Pan, Shuyi Yang et al.. Why Do Multi-Agent LLM Systems Fail?. 2025. arXiv:2503.13657. [Paper] [Code] [Data]

[7] Yanki Margalit, Nurit Cohen-Inger, Erni Avram et al.. Governed Shared Memory for Multi-Agent LLM Systems. 2026. arXiv:2606.24535. [Paper] [Code]

[8] LangChain. Deep Agents Overview. 2026. Official software documentation; accessed 2026-10-27. [Docs] [Code]

[9] Yuanzhe Hu, Yu Wang, Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. 2026. arXiv:2507.05257. [Paper] [Code] [Data]

[10] Hanxiang Chao, Yihan Bai, Rui Sheng et al.. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?. 2026. arXiv:2605.06527. [Paper] [Code] [Data]

[11] Adam Fourney, Gagan Bansal, Hussein Mozannar et al.. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. 2024. arXiv:2411.04468. [Paper] [Code]

[12] Yuzhe Yang, Yifei Zhang, Minghao Wu et al.. TwinMarket: A Scalable Behavioral and Social Simulation for Financial Markets. 2025. arXiv:2502.01506. [Paper] [Code]

[13] Bingquan Zhang, Xiaoxiao Liu, Yuchi Wang et al.. Human or LLM as Standardized Patients? A Comparative Study in Medical Education. 2025. arXiv:2511.14783. [Paper] [Code]

[14] Wujiang Xu, Zujie Liang, Kai Mei et al.. A-Mem: Agentic Memory for LLM Agents. 2025. arXiv:2502.12110. [Paper] [Code]

[15] Prateek Chhikara, Dev Khant, Saket Aryan et al.. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. 2025. arXiv:2504.19413. [Paper] [Code]

[16] NLP Course, CUHK-Shenzhen. CSC5051/CSC6052/MDS5110 Natural Language Processing, Fall 2026. 2026. Course website; accessed 2026-10-27. [Source]

[17] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov et al.. Evaluating Very Long-Term Conversational Memory of LLM Agents. 2024. arXiv:2402.17753. [Paper] [Code]